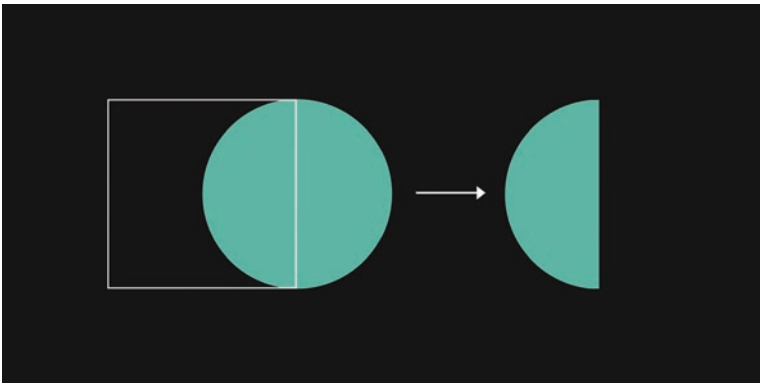

Figure 4-7. *How a clipPath works*

A `clipPath` requires an `id` so that it can be referenced later. Nested within the `clipPath` element is the actual shape or path that will do the "clipping." Here's how we'd create something like the semi-circle example from Figure 4-7.

```
// Initialise the clipPath with an id.
let clip = svg.create('clipPath').set({ id: 'clip1' });

// Create the shape of the clipPath.
clip.create('rect').set({
  x: 0, y: 0, width: 100, height: 100, stroke: '#fff'
});

// Apply it to a circle via a call to its id.
svg.create('circle').set({
  cx: 100, cy: 50, r: 50, fill: '#80ffe6', clip_path:
  'url(#clip1)'
});
```

As you can see, the `clipPath` is called via its `id`, treated as an internal `url` reference, much like gradients and patterns. Let's get back to our sketch now and apply this same logic within our loop. First, create the nested loop as we did before:

```
for (let y = 0; y < gridSize; y += increment) {
  for (let x = 0; x < gridSize; x += increment) {
  }
}
```

Next, create the `clipPath`. We need to create a unique `id` for each instance, so we can't pass in a static string as we did previously – otherwise, the `id` would be duplicated on each loop iteration. Instead what we can do is use the `x` and `y` iterator variables to generate a dynamic string, using template literal syntax like so:

```
// Create our clip path with a unique id.
let clip = svg.create('clipPath').set({ id: `${x}${y}` });
```

Now we can create the actual shape of the `clipPath`. We'll keep it straightforward and make it square, that is, a `rect` with a `width` and `height` equal to our `cellSize`.

```
// Create the clip path shape.
clip.create('rect').set({
  x: x, y: y, width: cellSize, height: cellSize
});
```

With the `clipPath` in place, we need to decide what to put into it. Here's where we can get creative! There are four corners of each cell, so what we'll do is randomly center a group of circles at any one of these four positions. The coordinates of these positions are determined by the current `x` and `y` values, along with the `cellSize`. Here's how we'd calculate these positions and choose one of them at random:

```
// Define our possible positions.
let positions = [
  [x, y], // top left
  [x + cellSize, y], // top right
  [x + cellSize, y + cellSize], // bottom right
  [x, y + cellSize] // bottom left
];

// Pick a random position.
let pickedPosition = Gen.random(positions);
```

Now, to create the `circle` group. This will involve a third loop (so a loop within a loop within a loop – loopception if you will), where we will create five circles radiating out from a randomly chosen center point. And let's not forget about our palette picked out earlier; the fill of each circle will be colored accordingly.

```
// Create a group for our circles.
let circles = grid.create('g');

// Create the circles, applying the picked position and palette.
for (let i = 0; i < 5; i += 1) {
  circles.create('circle').set({
    cx: pickedPosition[0],
    cy: pickedPosition[1],
    r: cellSize - (i * (cellSize / 5)), // this took a bit of
                                           tweaking
    fill: pickedPalette[i]
  });
}
```

We have a couple more steps to go. We haven't actually *applied* the `clipPath` to anything yet, so if we were to run our sketch at this point, we'd just see a fairly cluttered overlap of circles. We'll fix this by applying the

clipPath to the circle group rather than the circles themselves. And as the last step within the loop, we'll create a square border to frame each cell, just to give things a little more definition.

```
// Apply the clip path to the circle group.
circles.set({
  clip_path: `url(#${clip.get('id')})`
});

// Create a square to frame the cell.
grid.create('rect').set({
  x: x, y: y, width: cellSize, height: cellSize,
  fill: 'none', stroke: '#eee',
});
```

Now, outside the loop, center our grid as we've done before.

```
// Centre the grid within the viewBox.
grid.moveTo(500, 500);
```

And voila! You should now see some variations similar to those of Figure 4-8. (As an aside, if you find that the grid isn't always centering properly, that's because the grid's bounding box includes the invisible clipped content, which won't always arrange itself symmetrically around the grid.)

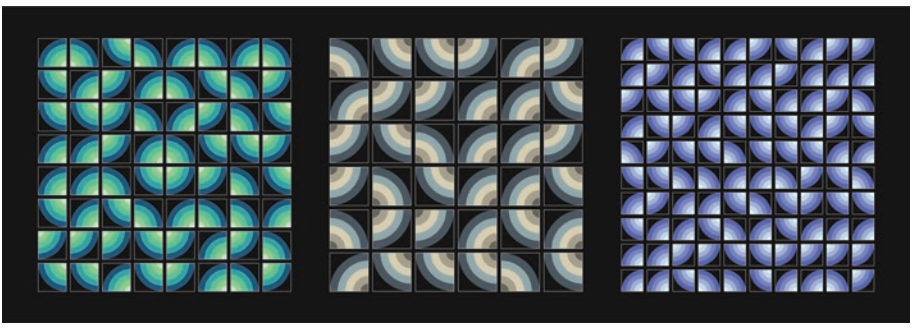

Figure 4-8. *Colorful clip path grid patterns*

Choice and Chance

What if we were to bring an element of chance into choosing whether or not to populate a given cell in our grid? This can lead to a less rigid-looking composition – which isn't necessarily better, but it can offer more in the way of variety.

The SvJs Gen.chance( ) Function

When we talk about probability, what we're usually trying to establish are the chances, or the odds, of something occurring. This something will either happen or it won't, so what we want is a binary response – or boolean – to tell us yes or no, win or lose. Will your horse Victor Vector give you a payout at the races today? Given the odds are 7 to 2, there's a 22% chance of some winnings (2 / (7 + 2) x 100 = 22.22).

How could we work this kind of logic into our compositions? It's actually quite simple to do with plain JavaScript and `Math.random()`, but SvJs offers a more intuitive, less verbose alternative with its `Gen.chance()` function.

By default (i.e., without any arguments), `Gen.chance()` returns either `true` or `false` based on odds of 50/50, or 50%. If we supply a single argument, it's interpreted as a percentage.

```
// The below returns true 60% of the time.
Gen.chance(60);
```

If two numbers are supplied, the arguments are interpreted as odds. This can be useful if you prefer thinking of probabilities more in terms of placing a bet at a bookie than calculating a precise percentage value (which is actually done for you under the hood).

```
// There is a 7 to 2 chance of this returning true.
Gen.chance(7, 2);
```

Chance in Action

Let's create a new sketch, using `Gen.chance()` to determine whether or not the cells within our grid get populated with content. Copy the `08-colourful-grids` folder and rename it to `09-chance`, and then remove all content from the inner loop (leaving you with an empty nested `for` loop). Next, replace the `palettes` array and `pickedPalette` variable with a randomly generated `hue`.

```
// Pick a random hue.
let hue = Gen.random(0, 360);
```

You can leave the grid-related variables as they are, or tweak them if you prefer. Next, *between* the `x` and `y` loops (so within the first loop but before the second), increment the `hue` like so:

```
// A nested loop to create the grid.
for (let y = 0; y < gridSize; y += increment) {

  // Increment the hue relative to the rows, keeping it within
     0 and 360.
  hue = (hue >= 360) ? (hue - 360) + (120 / rows) : hue + (120
  / rows);

  for (let x = 0; x < gridSize; x += increment) {
  ...
```

This might seem like an overly complex way of increasing the `hue` value, but really it's just doing two things: first, it's ensuring the `hue` value doesn't stray out of bounds (i.e., beyond `360`), and second, it's incrementing the `hue` value relative to the number of rows. This is so that sketch variations with a lower row count have a higher increment value, and those with a higher row count have a lower increment value, resulting in greater color consistency overall.

Inside the `x` loop is where we'll actually use `Gen.chance()`. Used as a condition inside an `if` statement, we can conditionally fire a third loop based on the result it returns. This loop will create random `line` elements inside each cell, but only if the cell is activated by the `Gen.chance()` function. We'll set the actual probability to 60%.

```
// Run the loop based on chance.
if (Gen.chance(60)) {
  for (let i = 0; i < cellSize; i += 1) {
    grid.create('line').set({
      x1: Gen.random(x, x + cellSize),
      y1: Gen.random(y, y + cellSize),
      x2: Gen.random(x, x + cellSize),
      y2: Gen.random(y, y + cellSize),
      stroke: `hsl(${hue} 80% 80% / 0.33)`
    });
  }
}
```

This should result in gradiented tetris-like arrangements, similar to Figure 4-9.

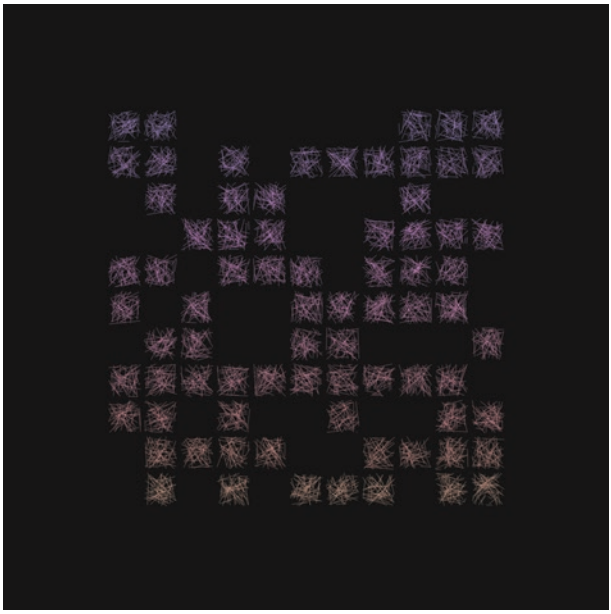

Figure 4-9. *Gen.chance() in action*

Probability Distributions

Probability distributions describe how numbers are spread out (or distributed) over a given range. Unless you've studied statistics in one form or another, you may not be too familiar with the term. And if the mere mention of statistics causes you to break out in a cold sweat, don't worry – we won't be delving into the underlying mathematics. We've actually been using a particular kind of probability distribution already but just haven't attached the name to it.

Uniform Distribution

The `Gen.random()` function is an example of *uniform* distribution, where every number within the range has an equal chance of being chosen. In other words, there are no biases or tendencies hiding with the

`Gen.random()` function (or indeed, the `Math.random()` on which it is based) that make it more likely that a certain subset of numbers within the range will be selected.

In Figure 4-10, we can see uniform distribution visualized as the variation of the `x` coordinate in the placement of 1000 vertical lines. The lines are all scattered with equal chance of appearing anywhere along the x axis.

Figure 4-10. *Uniform distribution of vertical lines*

Gaussian Distribution

In real-life measurements (i.e., those outside the binary world of ones and zeros), uniform distribution isn't actually that common. Far more prevalent is something called *Gaussian* distribution, where the majority of values cluster toward a midpoint and drop off either side of this. It is also known as normal distribution, and when graphed, it forms a bell-shaped curve. Visualized as a series of vertical lines, it looks like Figure 4-11.

Figure 4-11. *Gaussian distribution of vertical lines*

The midpoint around which most values accumulate is known as the mean, and the degree to which they drop off, or deviate, from this mean is known as sigma, or the standard deviation.

SvJs comes with a function called `Gen.gaussian()` that returns a random number that adheres to a Gaussian distribution. With no arguments, it will return values between approximately -3 and +3, with a mean of 0 and a standard deviation of 1. What this means in practice is that

- 68% of values will be within -1 and 1 (the standard deviation)
- 95% of values will be within -2 and 2
- 99% of values will be within -3 and 3

By supplying arguments to `Gen.gaussian()`, we can modify the mean and standard deviation.

```
// Adjusting the mean and standard deviation to 10 and 2
respectively.
Gen.gaussian(10, 2);
```

In the preceding example, adjusting the mean to 10 and standard deviation to 2 would translate to 68% of values falling within -2 and +2 of the mean (so between 8 and 12), 95% falling between 6 and 14, and more than 99% falling between 4 and 16. Strictly speaking, the values of a Gaussian distribution aren't bounded, so there may be occasional extreme outliers. If, for example, you wanted to make sure you kept the results to within a factor of 3 either side of the standard deviation, you could use the `Gen.constrain()` function to bound the results like so:

```
// Bounding the results of Gen.gaussian() to within -3 and +3.
let gaussian = Gen.gaussian(0, 1);
let constrainedGaussian = Gen.constrain(gaussian, -3, 3);
```

Let's move on and create a quick sketch to illustrate how we could use the results returned from `Gen.gaussian()` to map `x` and `y` coordinates across our `viewBox`. Copy our template and call it something like

`10-gaussian-dist`. In the usual place below our background, initialize a loop that will run 10,000 times (yep, that's a lot!), and inside it, generate a couple of Gaussian coordinates.

```
// Run a loop 10,000 times.
for (let i = 0; i < 10000; i += 1) {

  // Generate x and y co-ordinates with a gaussian distribution.
  let gaussianX = Gen.gaussian(500, 150);
  let gaussianY = Gen.gaussian(500, 150);
}
```

The coordinates are based around the center of the `viewBox` (i.e., 500), with a standard deviation of 150. Next, and still within the loop, create our lines, basing them on our Gaussian coordinates and adding a bit of randomization to their endpoints.

```
// Create the lines based on the gaussian co-ordinates.
svg.create('line').set({
  x1: gaussianX,
  y1: gaussianY,
  x2: gaussianX + Gen.random(-10, 10),
  y2: gaussianY + Gen.random(-10, 10),
  stroke: `hsl(${Gen.random(150, 270)} 80% 80% / 0.8)`
});
```

You could stop here and see the circular pattern that emerges, but I added another little loop (after the first one, not within it) to subtly emphasize the shape of the distribution with a series of fading circular strokes.

```
// Create a series of circles to frame the distribution.
for (let i = 0; i < 10; i += 1) {
  svg.create('circle').set({
    cx: 500, cy: 500, r: 25 + (i * 25), fill: 'none',
```

```
    stroke: `hsl(0 0% 0% / ${0.25 - (i / 50)})`, stroke_
    width: 15
  });
}
```

After this, you should end up with something similar to Figure 4-12.

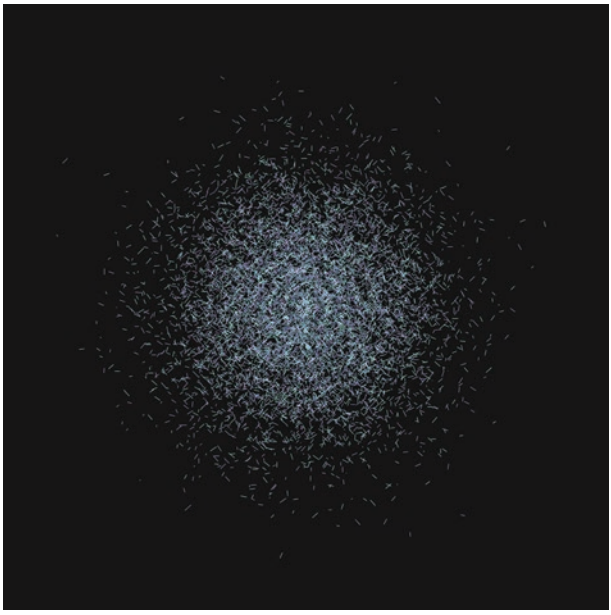

Figure 4-12. *Using Gen.gaussian to position 10,000 lines*

In terms of the generated markup, it's quite a heavy piece compared to anything we've done previously (10,000 elements will do that). This can adversely impact render performance and also means a bigger SVG file if you copy it from the HTML. One easy way to get the size down somewhat is to set the third argument to `false` in each instance of the `Gen.gaussian()` function. Like `Gen.random()`, `Gen.gaussian()` accepts a third argument that determines whether it returns a float (more accurate but with a lot of digits) or an integer (whole number). Dealing in whole numbers decreases the generated markup by roughly a factor of two in this case.

Pareto Distribution

Pareto distribution, also known as the Pareto Principle or the 80-20 rule (illustrated in Figure 4-13), is the final probability pattern we'll cover. It is named after an Italian economist who famously observed that just 20% of a society's population controlled 80% of its wealth. This was the 1890's mind, so had Pareto made the same observations today, we might be discussing the 99-1 rule! Be that as it may, the basic idea remains the same: there is a lot with a little and a little with a lot.

Figure 4-13. *Pareto distribution of vertical lines*

This can be useful in generative art to achieve a balance of differently sized elements. We'll use this in our next sketch to create a pseudo-cityscape called Porto Pareto, where the size of the city's buildings will be varied using the SvJs `Gen.pareto()` function. This function takes two arguments; the first defines the minimum number in the range to be returned, and the second defines whether this number will be a float or integer.

```
// Return a pareto-distributed integer, not less than 20.
Gen.pareto(20, false);
-> 32
```

Copy the template folder and name it `11-porto-pareto`, and below the background, create a group that will contain our cityscape.

```
// Create a group for our generative city.
let portoPareto = svg.create('g');
```

The cityscape will have three main elements: the sky, the river (or port), and the buildings. The sky and river will both be simple `rect` elements with gradients; we'll create these first.

```
// Create the sky gradient.
svg.createGradient('sky', 'linear', ['#f58b10', '#d21263',
'#940c5e', '#25226c'],
90);

// Create the sky and apply the gradient.
portoPareto.create('rect').set({
  x: 150, y: 150, width: 700, height: 400, fill: 'url(#sky)'
});

// Create the river gradient.
svg.createGradient('river', 'linear', ['#80e5ff10',
'#70b566'], 90);

// Create the river and apply the gradient.
portoPareto.create('rect').set({
  x: 150, y: 555, width: 700, height: 295, fill: 'url(#river)'
});
```

Hopefully there won't have been any surprises in the aforementioned. Next up, we'll deal out some Pareto distribution and create our buildings. In the following loop, there's one line in particular I want to unpack, and this is where we set the height using the `Gen.constrain()` function. Without it, our Pareto distribution would generate some dizzyingly high buildings that would extend well beyond the canvas limits. However, using `Gen.constrain()` alone with a set upper limit leads to a clipped effect on the highest buildings, so to even this out, I've added in a `Gen.random()` function to vary the upper limit.

```
// A loop for our generative cityscape.
for (let i = 0; i < 60; i += 1) {
  // Get a pareto distribution with a min height of 20.
  let pareto = Gen.pareto(20);

  // Constrain the height, and slightly randomise the upper limit.
  let height = Gen.constrain(pareto, 20, Gen.random(150, 200));

  // Create our buildings.
  portoPareto.create('line').set({
    x1: 150 + (i * 12), y1: 550,
    x2: 150 + (i * 12), y2: 550 - height,
    stroke: '#181818', stroke_width: 8
  });
}
```

We now have our cityscape in place, but we're not done yet. It's quite flat-looking, and I'd like it to be framed within a circle and for the colors to look like they're emanating outward. To achieve this, I'm going to use something called a `mask`.

Masking Our Content

A `mask` is very similar to a `clipPath`, except that it allows for degrees of transparency. A `clipPath` is an all-or-nothing affair; you're either inside or outside the bounds of the shape that defines it. With a `mask`, you can fade content in or out depending on the brightness or luminance of the shape. For this reason, it's most straightforward to use black and white when creating masks. White (`#ffffff`) translates to a fully transparent region, whereas black (`#000000`) translates to a fully opaque region.

In Figure 4-14, the triangle is the `mask` shape, and the result of applying it to the `rect` is shown on the right.

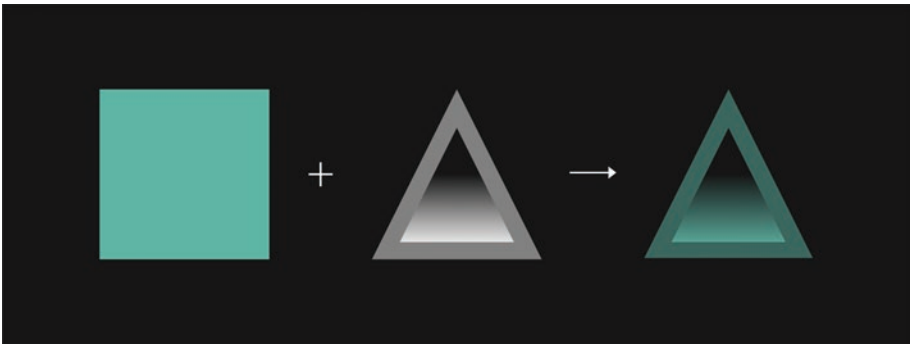

Figure 4-14. *How masks work*

As with a `clipPath`, the `mask` element requires an `id` and is applied to another element via a url reference to this `id`. What we're going to do is create a mask with a `circle` shape, and the fill of this `circle` will contain a radial gradient. We'll then apply it to the `portoPareto` group we created earlier.

```
// Create a radial gradient.
svg.createGradient('radialGrad', 'radial', ['#ffffff',
'#ffffff60']);

// Create a mask, and inside it create the circle with the
radial gradient.
let mask = svg.create('mask').set({ id: 'mask' });
mask.create('circle').set({
  cx: 500, cy: 500, r: 325, fill: 'url(#radialGrad)',
});

// Apply the mask to the group.
portoPareto.set({ mask: 'url(#mask)' });
```

You should now see the cityscape cut out in the shape of this circle, with the edges slightly fading out. As a final step, I've added in a gradiented circular stroke to frame the content a little more clearly.

```
// Create a linear gradient for our circular frame.
svg.createGradient('strokeGrad', 'linear', ['#eeeeee',
'#eeeeee15']);

// Create the frame and apply the gradient.
svg.create('circle').set({
  cx: 500, cy: 500, r: 345, fill: 'none',
  stroke: 'url(#strokeGrad)', stroke_width: 2.5
});
```

A variation of the result is shown in Figure 4-15.

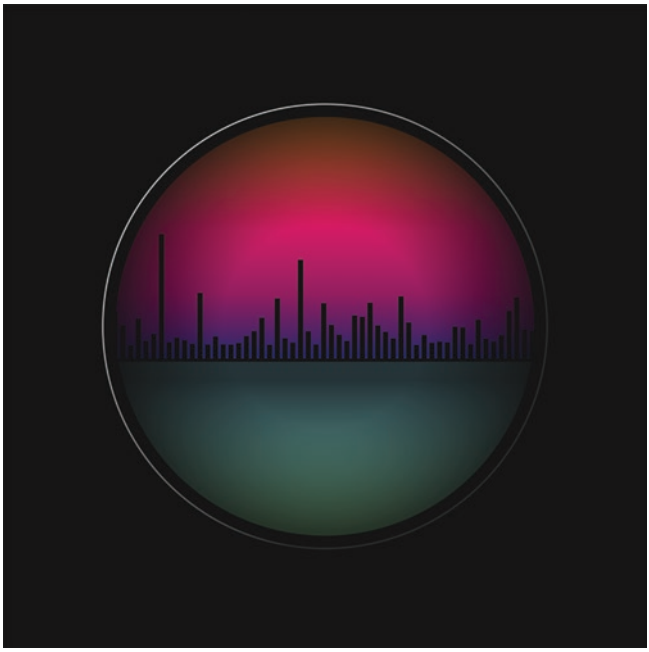

Figure 4-15. *Our Porto Pareto cityscape*

Summary

In this chapter, we've covered the following:

- Randomizing coordinates, colors, element dimensions, and more
- How to randomly pick items from single and two-dimensional arrays
- Regular grid construction with nested `for` loops
- How to use clipPaths and masks
- Using chance as a way to trigger element creation
- How to work with Gaussian and Pareto probability distributions

In the next chapter, we'll learn how to use noise to add a more organic kind of randomness to our compositions.

CHAPTER 5

The Need for Noise

In this chapter, we'll be covering the limitations of the random functions we've utilized thus far and how these limitations can be addressed by using the SvJs `Noise` module. We'll be exploring what noise is and the uses to which it can be put, and by the end, we'll have added a significant technique to our generative arsenal.

Random Limits

Sometimes randomness is just too, well … random. Even when we shape randomized values with probability distributions and constrain them within structured patterns, the variance between consecutive values invariably has that characteristic "staccato" feel. In other words, the values will always be somewhat jumpy and jittery when looked at side by side. You won't see a smooth progression from one value to another.

Don't get me wrong – this is often what we want, and it's what random functions are designed to do. But it can be tricky to create anything that has an organic quality to it with randomness alone. For this, we need noise.

Making Noise

In 1997, Ken Perlin won an Oscar. What made this award unprecedented was that Perlin wasn't an actor, director, or soundtrack composer, but a programmer. For the movie *TRON*, released in 1982, Perlin had been

D. Matthew, *Generative Art with JavaScript and SVG*, Design Thinking,
https://doi.org/10.1007/979-8-8688-0086-3_5

tasked with the development of procedural textures that would make 3D objects look more natural (i.e., organic), and the algorithm he devised, named Perlin noise, has since become ubiquitous in the world of computer graphics. You'll find implementations in image editors like GIMP and Photoshop, in vector editors like Inkscape and Illustrator, in 3D graphics packages like Blender, and in countless video games down through the ages. Minecraft is a great example; those infinite blocky terrains where players mine for resources use height maps generated by Perlin noise to determine their various peaks and troughs.

So Perlin noise is everywhere in the graphical domain. But what exactly is it?

Noise Explained

Technically, noise functions do produce random numbers, but they do so in a smoothly ordered fashion, and this is what makes the difference. In Figure 5-1, the height of the white lines is randomized using a Perlin noise variable. This is an example of one-dimensional noise, that is, noise values that operate along a single axis.

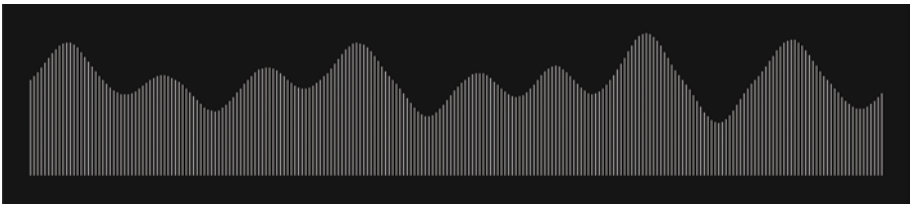

Figure 5-1. *A one-dimensional representation of Perlin noise values*

Noise is by nature multidimensional; Perlin originally designed it for 3D spaces, and there are four-dimensional implementations out there too, but with SVG being a two-dimensional medium, I'll be sticking to the x and y axes. The particular algorithm Perlin developed works by blending randomized values in a gradiented fashion over a given area. This area, or space, is more notional than actual; the noise values don't exist on an actual plane or separate canvas, but rather in an abstract (and infinite) space that we can retrieve values from.

For example, fetching a noise value at the conceptual coordinates of (15, 20) would give us some value between -1 and 1. And when we slowly traverse through this noise space to reach an adjoining coordinate, say (16, 21), we get a series of smoothly varied values we can work with. And it's what we can do with those values that make noise interesting from an aesthetic perspective.

In Figure 5-2, we can see noise represented in two dimensions using the full area of a canvas. This example is actually pixel based rather than vector based, chosen because it's perhaps the best way to visually grasp the notion of a noise space. Here, the alpha component of each pixel is varied according to the corresponding noise value, creating a cloud-like formation. You can immediately see why it would be useful for creating textures.

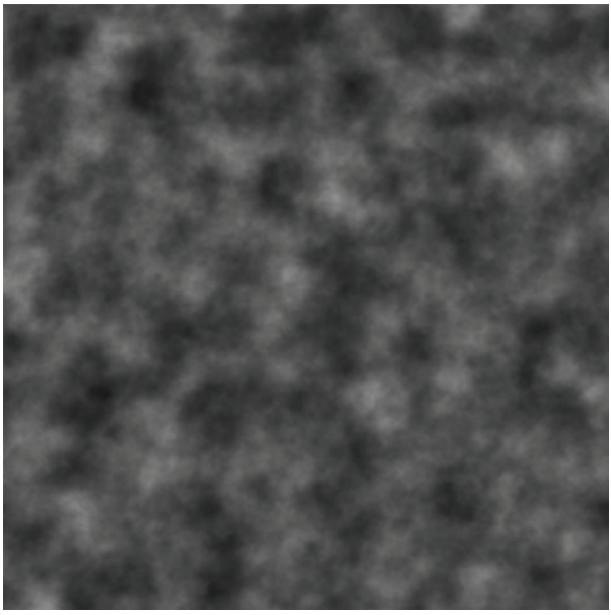

Figure 5-2. *A two-dimensional representation of Perlin noise values*

The SvJs Noise Module

The SvJs library comes with its own implementation of noise, designed to be simple to use and to have as light a footprint as possible. It resides in its own module, so like the `Gen` module, we have to import it before we can use it.

```
import { Noise } from '../../node_modules/svjs/src/index.js';
```

Unlike `Gen`, which is a collection of functions, the `Noise` module is class based, so we first need to instantiate it.

```
// Create an instance of the Noise class.
let noise = new Noise();
```

The class has a single method called `get()` that accepts an x coordinate and optional y coordinate as arguments and returns the value at that point in the noise space.

```
let noiseValue = noise.get(noiseX, noiseY);
```

That `noiseValue` won't be much use to us as a static value; what we need to do is gradually modify it by traversing the noise space. We do this by incrementing the `noiseX` and/or `noiseY` coordinates. Here's how this might look within a loop:

```
// Initialise our noise instance and noise co-ordinates first.
let noise = new Noise();
let noiseX = 15, noiseY = 20;

// Run the loop, slowly incrementing the co-ordinates to modify
the value.
for (let i = 0; i < 1000; i += 1) {
  let noiseValue = noise.get(noiseX, noiseY);
  ... // Do something with noiseValue

  noiseX += 0.003;
  noiseY += 0.003;

}
```

As you can see, we define the initial `noiseX` and `noiseY` coordinates outside the loop, use them to fetch a `noiseValue` within it, and then before the loop runs again, increment the coordinates by a very small amount. This amount we might refer to as the noise speed or rate of change. It's important when working with noise that this rate of change is minimal; comparatively large leaps (e.g., by whole numbers) won't yield usable results.

Into the Noise Matrix

Let's copy our template and set up a new sketch folder called `12-noise-matrix`, to show how noise can be used to alter color values within a grid. At the top of our `sketch.js` file, add in the `Noise` module to our import statement like so:

```
import { SvJs, Gen, Noise } from '../../node_modules/svjs/src/
index.js';
```

Next, we're going to shorten the expression that calculates the `svgSize` (and thus the dimensions of our `viewport`). Instead of the ternary operator, we can actually use the `Math.min()` function, which returns the smallest of two or more values. If you remember, all we want to do is set the `svgSize` to whichever of the window's `innerWidth` or `innerHeight` is smallest, and `Math.min()` does this in the most succinct manner. A small tweak to be sure, but one that hints at the many uses of JavaScript's built-in `Math` module.

```
const svgSize = Math.min(window.innerWidth, window.
innerHeight);
```

I'd recommend applying this tweak to our base template too.

A Noisy Grid

For this composition, we'll be using a familiar technique: we'll be setting up a grid, and this grid will act as our matrix (a matrix is, after all, just a grid of values). And the matrix we'll be creating in this sketch will mimic the one from the movie of the same name. We'll be populating the grid with ones and zeros, and the color of these digits will be modulated by noise to achieve that cascading, desaturated effect.

Create an instance of the noise class, and then initialize three further variables. Two will take care of our noise x and y coordinates, and in the third, we'll store the noise speed that will determine the rate at which we'll increment the noise coordinates.

```
// Create our noise, the noise x and y co-ordinates, and
noise speed.
let noise = new Noise();
let nX = 0, nY = 0;
let noiseSpeed = 0.5;
```

The next block of code should hopefully look familiar. We're setting some grid-related variables, and this time we'll omit the `spacing` and `cellSize` variables we used in previous sketches and set the `gridSize` to cover the entire `viewBox`.

```
// Set some grid-related variables.
let noiseGrid = svg.create('g');
let gridSize = 1000;
let rows = 80;
let increment = gridSize / rows;
```

Next comes our loop. Instead of starting with the y coordinate in the outer loop, we're going to start with the x coordinate. What this will do is construct our grid from top to bottom and then left to right. In other words, we'll fill out our columns first rather than the rows, which will allow our noise values to flow in a downward motion.

Within the loop, we'll fetch our noise value using the `noise.get()` method, passing in the `nX` and `nY` values we initialized earlier.

```
// Create the noise matrix.
for (let x = 0; x < gridSize; x += increment) {
  for (let y = 0; y < gridSize; y += increment) {
```

```
    // Fetch the noise value.
    let noiseValue = noise.get(nX, nY);

  }
}
```

Next, let's create the actual text content. We want a series of ones and zeros to fill the grid, with a 50% chance of either digit being chosen. As we learned in the last chapter, the `Gen.chance()` function is the simplest way to do this, combined with a ternary operator.

```
// Create text displaying either 0 or 1 (50% chance).
let text = noiseGrid.create('text');
text.content(Gen.chance() ? '1' : '0');
```

Now we need to set the position of the digit with respect to the `x` and `y` loop iterator values and set the intensity of the fill color (i.e., the lightness component of the `hsl()` function) with the `noiseValue` variable. We should also set a font size and font family while we're at it.

```
text.set({
  x: x, y: y,
  font_size: 16,
  font_family: 'serif',
  fill: `hsl(120 20% ${noiseValue}%)`
});
```

And before we close out our loop, we'll want to increment our `nX` and `nY` values with the `noiseSpeed` variable.

```
nX += noiseSpeed;
nY += noiseSpeed;
```

To finish off the sketch, outside the loop, call the `moveTo()` function to shift the grid to the center of the canvas.

```
// Centre the grid within the viewBox.
noiseGrid.moveTo(500, 500);
```

You should now see a grid of zeros and ones when you run your sketch. But … they're all black. The color lightness isn't visibly changing, even though we're using our `noiseValue` to modify it. What's going on here?

If you inspect some of the `text` elements in the browser console, you'll see something like the following:

```
<text x="860" y="960" font-size="16" font-family='serif'
fill="hsl(120 20%
-0.11888951723051341%)">0</text>
<text x="860" y="980" font-size="16" font-family='serif'
fill="hsl(120 20%
-0.06168012594419729%)">1</text>
<text x="880" y="0" font-size="16" font-family='serif'
fill="hsl(120 20%
0.06179162569500480 7%)">0</text>
<text x="880" y="20" font-size="16" font-family='serif'
fill="hsl(120 20%
0.1202821001266494%)">1</text>
```

The point to note here is that when we call the `noise.get()` method, the value returned is a float between -1 and 1 (with some occasional outliers). The lightness component of the `hsl()` function, however, should have values between 0 and 100%, so in its raw state, the noise value isn't very useful here. How might we fix this? Couldn't we just multiply it by a factor of 100? We could, but we'd then have an equal amount of numbers in the negative range. And if we tried to remedy this with `Math.abs()` to force the negative numbers into a positive range, that would destroy the very raison d'être of noise – its smoothly transitioning values. So what's the solution?

Mapping the Noise Values

This is actually a common challenge in generative art – mapping a series of values from one range to another. Thankfully the Gen module comes equipped with a function that does just this: `Gen.map()`.

The function requires five arguments, with an optional sixth: the first is the value to be mapped to the new range, the second and third arguments define the lower and upper bounds of the existing range, and the fourth and fifth arguments define the lower and upper bounds of the range to which we want our value mapped. The final sixth argument specifies whether a float should be returned. In common with other Gen functions like `gaussian` and `pareto`, this is set to `true` by default, and setting it to `false` rounds the result to the nearest integer. The following is an example:

```
// Map a number (5) from one range (0, 10) to another (0, 100).
let num = 5;
num = Gen.map(num, 0, 10, 0, 100);
console.log(num);
-> 50
```

If we apply the `Gen.map()` function to our `noiseValue` variable, we can retrieve a more usable value. We could assign this new mapped value to a new variable, or as the preceding example shows us, we can simply reassign the original variable.

```
// Fetch the noise value.
let noiseValue = noise.get(nX, nY);

// Map the noise value to a useful range.
noiseValue = Gen.map(noiseValue, -1, 1, 0, 100, false);
```

Once you've done this, you should see some color injected into our matrix (as per Figure 5-3). Enough we hope to get an appreciative nod from Neo.

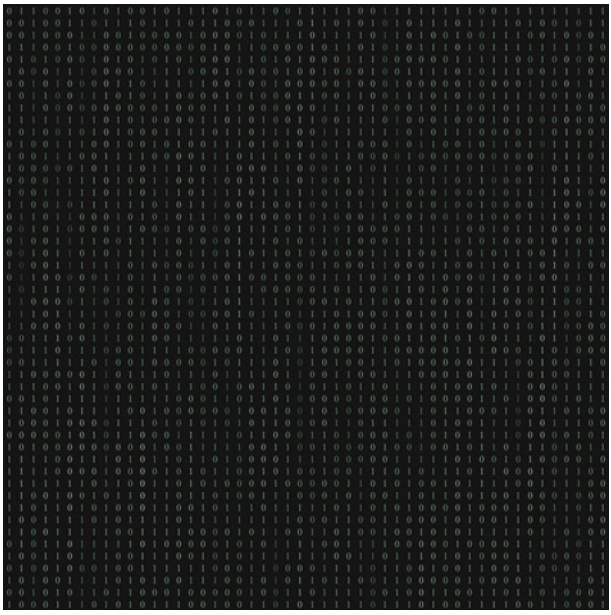

Figure 5-3. *That chunk of the matrix that tastes like chicken*

It's worth noting that when using a generic `font-family` like `serif`, you essentially let the operating system decide which default font to render. For this reason, what you see may differ slightly from Figure 5-3.

Optimize with Style

We could leave it there and make no further changes to the sketch, but there's one little niggle I can't leave alone. In total, we have 2,500 `text` elements in this sketch, and for each of these elements, the `font-size` and `font-family` attributes remain static throughout. This seems quite wasteful.

```
<text x="740" y="200" font-size="16" font-family="serif"
fill="hsl(120 20%
50%)">1</text>
```

What we can do is first remove the font attributes set within the loop so that the `text.set()` method looks like the following:

```
text.set({
  x: x, y: y, fill: `hsl(120 20% ${noiseValue}%)`
});
```

Then we can create a `style` element and within it, target all our `text` elements at once. Near the top of our sketch, after the parent SVG declaration, include the following code:

```
// Set some text styling.
svg.create('style').content(`
  text {
    font-size: 16px;
    font-family: serif;
  }
`);
```

Note the use of backticks here; this allows us to indent our code just as we would with normal CSS. What this CSS does is apply the `font-size` and `font-family` to each of our `text` elements, avoiding the use of repetitive inline attributes. This is not only a more elegant way of handling static styles, but it can save us quite a few kilobytes along the way. In this particular sketch, taking this measure resulted in a 40% reduction in the rendered markup – not bad!

Spinning Noise

In our next sketch, to consolidate some of the concepts covered already, we're going to spin some `line` elements about their center points and vary their line lengths and colors with noise. This time we'll use just a single loop and noise dimension.

Copy the previous sketch folder and name it something like `13-spinning-noise`. Remove the `style` element and everything after the background so that we're left with the following boilerplate code:

```
import { SvJs, Gen, Noise } from '../../node_modules/svjs/src/
index.js';

// Parent SVG.
const svg = new SvJs().addTo(document.
getElementById('container'));

// Viewport and viewBox (1:1 aspect ratio).
const svgSize = Math.min(window.innerWidth, window.innerHeight);
svg.set({ width: svgSize, height: svgSize, viewBox: '0 0 1000
1000' });

// Background.
svg.create('rect').set({
  x: 0, y: 0, width: 1000, height: 1000, fill: '#181818'
});
```

We'll then set up our noise-related variables, along with a couple of randomized values that will determine our initial hue and the amount of times our loop will run.

```
// Noise-related and randomised variables.
let noise = new Noise();
let nX = 0;
let noiseSpeed = 0.025;
let lines = svg.create('g');
let hue = Gen.random(0, 360);
let iterations = Gen.random(60, 100);
```

Mapping and Constraining

We can now run the loop. We'll offset our iteration start point by `10`; for this particular sketch, this value just worked better than starting at the usual `0`. Within the loop, we'll fetch the `noiseValue`, and then we'll extract two further variables from this by individually mapping the value to different ranges. One will control how much we shift the `hue`, and the other will control the length of each line.

```
// Start the dance.
for (let i = 10; i < iterations; i += 1) {
  let noiseValue = noise.get(nX);
  let hueShift = Gen.map(noiseValue, -1, 1, -180, 180, false);
  let lineLength = Gen.map(noiseValue, -1, 1, 0, 1000, false);
}
```

After the `lineLength` variable (and still within the loop), set up the first of our lines. We will start off with a straight vertical line at `(0, 0)` (the top left of our `viewBox`), and we'll worry about centering things later. As it's a vertical line, the second `x` coordinate won't change – only the second `y` coordinate will.

Next comes the `stroke` value. We'll wrap this in the `Gen.constrain()` function to keep the values between `0` and `360`, and inside this, the `hue` will increment by the `hueShift`. We'll then set both the `opacity` (within the `hsl()` function) and the `stroke-width` to `0.5`, which will keep the line nice and delicate.

```
let l1 = lines.create('line').set({
  x1: 0, y1: 0, x2: 0, y2: lineLength,
  stroke: `hsl(${Gen.constrain(hue + hueShift, 0, 360)} 80% 80%
  / 0.5)`,
  stroke_width: 0.5
});
```

For the second line, we'll do much the same, except this time we'll stretch the `lineLength` a little, shift the `hue` in the opposite direction, and also reduce its `opacity`.

```
let l2 = lines.create('line').set({
  x1: 0, y1: 0, x2: 0, y2: lineLength * 1.1,
  stroke: `hsl(${Gen.constrain(hue - hueShift, 0, 360)} 80% 80%
  / 0.25)`,
  stroke_width: 0.5
});
```

Rotating and Translating

The final step will involve some transforms and the usual incrementation of our noise coordinate. We'll rotate the first line positively in a clockwise direction and the second line negatively in an anticlockwise direction. For this, we'll use the SvJs `rotate()` method, which is simpler than setting a `transform: rotate()` attribute as we've done in some previous sketches. It also boasts some additional benefits:

- It rotates relative to its own center by default, rather than the top left of the canvas.
- It preserves any existing transforms, rather than overwriting them as the `transform` attribute does.

With that in mind, let's continue with the sketch and complete the loop.

```
l1.rotate(i);
l2.rotate(-i);

nX += noiseSpeed;
```

And finally, outside the loop, move the lines to the center of the canvas and rotate them by a random amount between 0 and 360.

```
lines.moveTo(500, 500);
lines.rotate(Gen.random(0, 360));
```

As you can see in Figure 5-4, what we end up are a series of slightly asymmetric lines that somewhat resemble, to me at least, the dance of some deep-sea creatures.

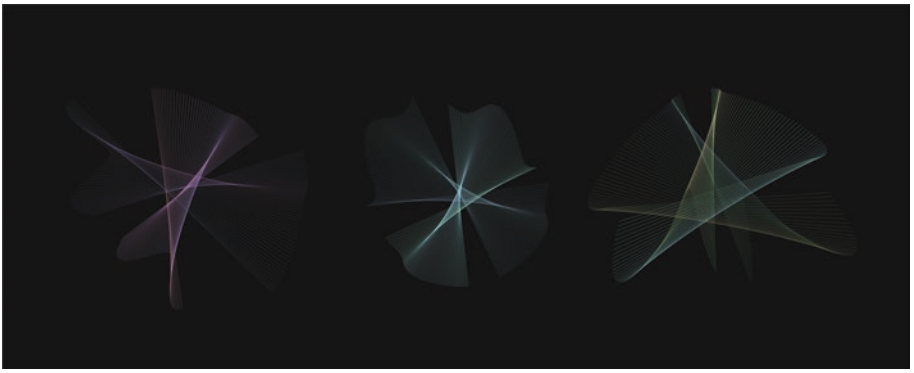

Figure 5-4. *Spinning lines or deep-sea creatures?*

Summary

While there's certainly more to explore where noise is concerned, we've covered the basics here, and we will be utilizing noise again in the forthcoming chapters. If you want to be able to tinker more with the nuts and bolts of noise, I'd encourage you to check out some third-party libraries like the popular SimplexJS, which goes beyond two dimensions and allows for greater freedom of configuration (at a cost of size, however).

Before we move on, let's quickly recap what we've covered in this chapter:

- The limits of randomness when creating more organic forms and how noise addresses this limitation
- The theory behind the abstract noise space
- How to put this into practice with the SvJs `Noise` module
- How to traverse the noise space within a loop
- How to use noise values to modify element coordinates and colors
- Mapping and constraining noise values to more useful ranges
- Optimizing repetitive element attributes with CSS
- Nondestructive element rotations

In the next chapter, we're going to cover a fundamental part of the SVG spec: the very powerful path element.

CHAPTER 6

The All-Powerful Path

Paths are perhaps the most important and most powerful part of the SVG spec. Most SVG files, be they simple icons or complex artworks, consist primarily of path data. If, for example, you draw anything more advanced than a primitive shape in a vector graphics program like Inkscape or Illustrator, you are ipso facto working with paths. And no, to program paths manually in JavaScript and SvJs, you don't need to be a mathematical wizard.

This chapter will cover the path element and its associated commands, which are sufficiently numerous to need their own subsections. Paths can get complicated quickly, so we'll also be covering some SvJs methods to make our lives easier.

The Path Element

It's simple to set up a path; we just call the SvJs `create()` method as we would with most other elements:

```
let path = svg.create('path');
```

Paths accept the usual `fill` and `stroke` attributes that other graphical elements do, but unlike other graphical elements (like `rect` and `circle`), they come with no intrinsic form or shape. This is entirely up to us to define, and we do so via the `d` attribute.

D. Matthew, *Generative Art with JavaScript and SVG*, Design Thinking,
https://doi.org/10.1007/979-8-8688-0086-3_6

D for Data

The `d` attribute allows us to populate a path with data. This data consists of a series of path commands and accompanying numeric values that define the form the path takes. The data as a whole is passed in as a string.

The creators of the SVG spec knew that paths would feature heavily in most SVG files, so creating concise path commands was a crucial part of keeping down the overall kilobyte count. They are therefore of single-character length, like the `d` attribute to which they are passed, and the values that accompany the commands can also be flexibly formatted to conserve space.

```
// An example of a string of path data.
path.set({
  d: 'M 20,20 L 30,20 L 30,30 L 20,30 Z'
});

// An identical path using a more concise string.
path.set({
  d: 'M20 20L30 20L30 30L20 30Z'
});
```

We'll go into more detail in later sections as to how examples like the aforementioned work; for now, just note that the very same path data can be written with or without commas to separate values and with or without spaces to separate commands.

Path Commands

You can think of path commands as drawing instructions. Each command is represented by a letter, and this letter can be either uppercase or lowercase. The uppercase version indicates that the values which follow will use absolute coordinates (so the command `M 50,50` would mean

moving to the exact position of `50,50` in the viewBox), whereas the lowercase version means that relative coordinates will be used (so the command `m 50,50` would mean moving by `+50` on both the x and y axes relative to the last known position).

There are a total of ten commands; twenty if you count the two versions. And if you think that sounds like a lot, you'd be right! It is a lot. But there's an amazing economy to these commands when you consider that they can be combined to create any possible two-dimensional shape. Some commands are more common than others, some more complex, and some we'll demonstrate the once and won't use again.

Starting and Ending a Path

All paths start somewhere. And where SVG is concerned, all paths start at the point to which we move our virtual pen. We use the `M` (or `m`) command for this, which means "move to." We follow this with an x and y coordinate pair to define the path's starting point. The `M` command by itself doesn't draw anything – it merely moves us to the point at which we want to begin. Our pen has yet to push down on the paper, so to speak.

```
// Syntax for the M/m command.
'M [x, y] ...'
'm [dx, dy] ...'

// Examples of the M/m command.
'M 50 100 ...'
'm 50 100 ...'
```

We can provide an optional `Z` command at the very end of a path to close it. Paths without a closing `Z` command remain open (i.e., the starting point and end point remain unconnected). `Z` will draw a straight line to

connect these two points unless the end point is a curve, in which case it will follow the curvature of the last control point (this will make more sense later on).

```
// Closing a path.
'M 50 100 ... 50 150 Z'
```

Unlike other path commands, the lowercase `z` and uppercase `Z` perform identical operations, as no values follow them. This means you can use them interchangeably.

Straight Lines

To ease us into path creation, we'll start with the straight line. There are actually a few different commands that can draw a straight line, but we'll start with the most obvious.

The Simple L

The `L` command allows us to draw straight lines defined by an x and y coordinate pair. The lowercase `l` does the same but uses coordinates relative to the path's last entered point.

```
// Syntax for the L/l command.
'L [x, y] ...'
'l [dx, dy] ...'
```

Let's see what this looks like in practice. In the following example, two lines are used to create two instances of the letter L. The first uses absolute coordinates; the second uses relative coordinates.

```
// The first L.
svg.create('path').set({
  d: 'M 300 200 L 300 800 L 600 800'
});

// The second L.
svg.create('path').set({
  d: 'M 675 825 l 0 -600 l -300 0'
});
```

We can see a rendering of the preceding code in Figure 6-1. I've left out any elements and attributes extraneous to the example and focused on just the path data. As you can see, you can use either of the `L` commands to achieve similar results.

When creating paths manually in this manner, some readers may find absolute coordinates more intuitive to work with, while others may find relative coordinates easier to use. Relative coordinates can be especially useful if, say, you wanted to move a path's location without disturbing its shape; with absolute coordinates, we'd have to change all the subsequent numbers, whereas with relative coordinates, we'd only have to change the initial `M` point.

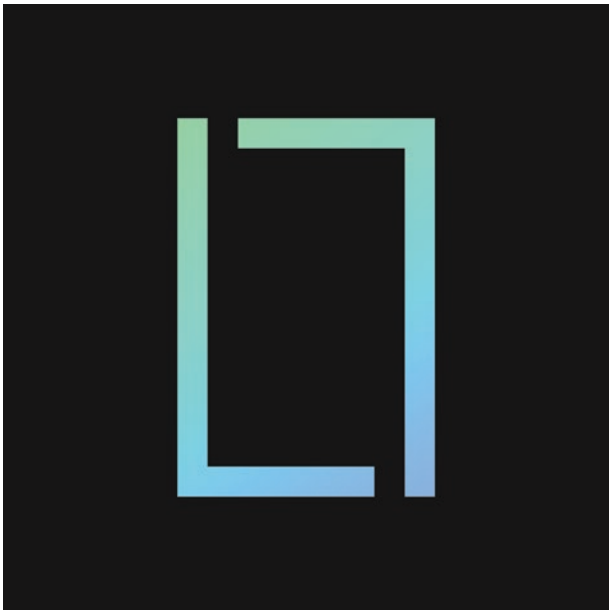

Figure 6-1. *Two L's drawn by two L commands*

Horizontal and Vertical Varieties

If the lines that we're drawing are either horizontal or vertical, we can use the `H` or `V` commands as a shortcut, as they only need the one parameter; in the case of `H` or `h`, the x coordinate, and in the case of `V` or `v`, the y coordinate.

```
// Syntax for the H/h command.
'H [x] ...'
'h [dx] ...'

// Syntax for the V/v command.
'V [y] ...'
'v [dy] ...'
```

These commands I don't find particularly useful for generative art, but it's worth knowing they're there should you want to use them. Let's create the same L shapes from Figure 6-1, but with H and V this time, including their lowercase versions.

```
// The first L.
svg.create('path').set({
  d: 'M 300 200 V 800 H 600'
});

// The second L.
svg.create('path').set({
  d: 'M 675 825 v -600 h -300'
});
```

Further Economies

Before we move on to curves, it's worth pointing out if we are using successive path commands of the same type, the command itself can be removed rather than repeated. SVG renderers are intelligent enough to know that if a path command is omitted, the last-known command should be used. Let's show a quick example (the output of which is shown in Figure 6-2).

```
// The first path, with repeating L commands.
svg.create('path').set({
  d: 'M 10 10 L 20 20 L 30 10 L 40 20 L 50 10 L 60 20 L 70 10'
});

// The second (identical) path, with the repeating L commands
omitted.
svg.create('path').set({
  d: 'M 10 10 L 20 20 30 10 40 20 50 10 60 20 70 10'
});
```

Figure 6-2. *Omitting repetitive commands*

Quadratic Bezier Curves

The first curve we'll cover is the quadratic Bezier curve. Don't worry – the rather daunting name doesn't reflect its complexity. As far as curves go, it's relatively simple. Two sets of coordinates are required: the first set defines the control point (which we'll explain in a moment), and the second the destination point.

```
// Syntax for the Q/q command.
'Q [cpx cpy x y] ...'
'q [dcpx dcpy dx dy] ...'
```

Control Points

If you imagine a control point as a force of attraction, or pull, toward which a curve bends, you'll have the right idea. Control points aren't themselves rendered, but if you look at the dot and dashed line in Figure 6-3, you can see a visualization of the force a control point exerts over its curve (in this case, a quadratic Bezier curve). In this example, the dot represents the control point coordinates of `[150, 350]`, given the following path:

```
'M 50 150 Q 150 350 250 150'
```

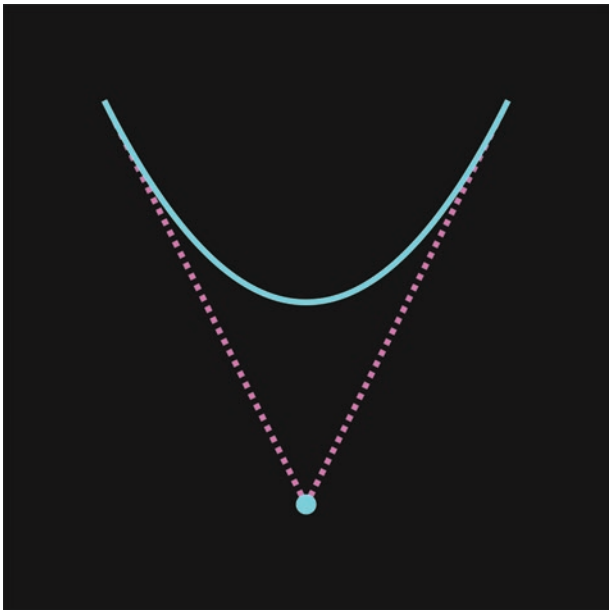

Figure 6-3. *The control point of a quadratic Bezier curve*

A quadratic Bezier curve has just the one control point, whereas the cubic Bezier curve (explored in a later section), has two.

A Smooth Shortcut

Say we wanted to extend the path from Figure 6-3 and draw a smooth set of waves repeating at regular intervals. The difficulty in doing this manually is with the calculation of the control point; if it's not perfectly symmetric with the previous one, a "kink" will appear in our curve (see Figure 6-4). Now, in this particular case, it's not that arduous a job to calculate a control point that would correct this, but in more complex cases, this can become a challenge.

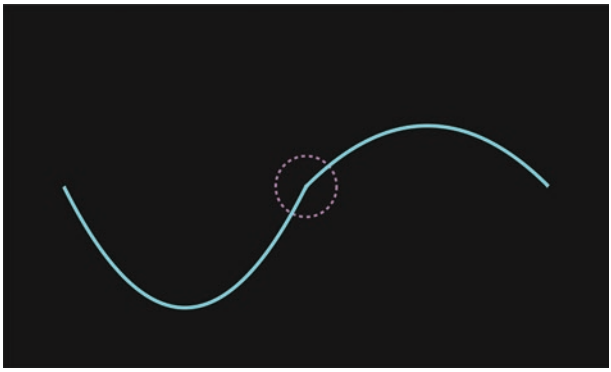

Figure 6-4. *A kink in the curve*

This is where the T command can help us. It extends a quadratic Bezier curve based on the previous curve's control point, essentially "mirroring" or reflecting it. The result is a smooth curve with a symmetric control point, and it requires just a single set of coordinates.

```
// Syntax for the T/t command.
'T [x y]'
't [dx dy]'
```

Let's extend the curve from Figure 6-4, adding a few extra undulations to it. In the following example code, I'm adding some commas to the coordinates for the sake of readability, but remember that they're not actually required. I'm also taking advantage of the fact that I don't need to repeat the T command when using it several times consecutively. This is why you'll only see it used once for the four coordinates that follow.

```
// Extending a quadratic curve with the smooth T command.
svg.create('path').set({
  d: 'M 50,150 Q 150,350 250,150 T 450,150 650,150 850,150
  1050,150'
});
```

Figure 6-5 is the result, with some circular highlights added in to distinguish the initial `M` point (yellow) and the full `Q` curve (the purple second point) from the four additional `T` points (in red).

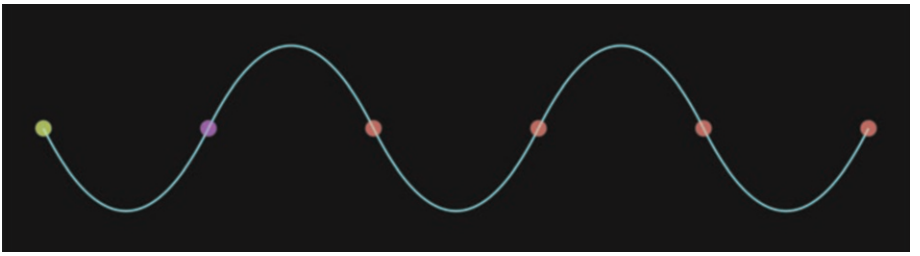

Figure 6-5. *A smooth quadratic curve*

A Quadratic Slinky

Let's use the quadratic Bezier curve in a simple sketch to create a Slinky-esque array of paths. If you don't know what a Slinky is, it's a popular spring-based toy that can "walk" itself down a set of steps if given a nudge from the top (I'm sure it can perform many other miraculous feats, but this is the one for which it's most widely known).

Copy the template folder and rename it to `14-quadratic-slinky`. Below the parent SVG, set some styles to target the `path` elements that we'll later set up as children of a group to which we'll give an `id` of `slinky`.

```
// Style the slinky.
svg.create('style').content(`
  #slinky path {
    fill: none;
    stroke-width: 0.75;
    stroke-linecap: round;
  }`
);
```

Next set up the usual background, initialize a random hue, and create the aforementioned `slinky` group.

```
// Background.
svg.create('rect').set({
  x: 0, y: 0, width: 1000, height: 1000, fill: '#181818'
});

// Choose a random starting hue.
let hue = Gen.random(0, 360);

// Set up the slinky path group.
let slinky = svg.create('g').set({ id: 'slinky' });
```

In the next step, we'll fire up the loop and create a couple of control points we'll use to shape our quadratic curve.

```
// Start the loop.
for (let i = 0; i < 500; i += 5) {

  // Create the control points.
  let cpx = Gen.random(200, 400);
  let cpy = i - 400;

}
```

The width of our curve will be 600 units (relative to the `viewBox`), so our halfway point will be 300. On the x axis, the `cpx` variable is picking a random integer value 100 units either side of the halfway point, to give our slinky some "wobble." The `cpy` variable is more straightforward; it moves down the screen on each iteration, with an offset of 400 units that will "pull" the curve upward.

Next, let's create the actual curve (keeping within the loop).

```
// Create the quadratic curve.
slinky.create('path').set({
  stroke: `hsl(${hue} 90% 80% / 0.85)`,
  d: `M 0 ${i} q ${cpx} ${cpy} 600 0`
});
```

Here, we're setting the `hue` as we've done many times before, and for the `path`, we're drawing one simple curve starting with the `M` command. For the curve coordinates, we're taking advantage of the relative coordinates calculated by the lowercase `q` command, which allows us to keep the values in step with curve's starting point.

Before closing out the loop, we'll increment the `hue`, but we'll use a new technique to do so. I mentioned the modulo operator (`%`) in Chapter 2 (in the section covering operators), which calculates a number's remainder after division. This can be useful to cycle within predefined ranges, such as the 0 to 360 constraints of a color's `hue`. Add the following as the final line before the end of the loop:

```
// Increment the hue.
hue = (hue % 360) + 1.5;
```

A quick explanation of how this works: say our random `hue` is initialized to 350. The expression `hue % 360` will return 350, because `350 / 360` is 0, with a remainder of 350. This works similarly for every number in our range *except* 360, which returns 1 when divided by itself and returns a remainder of 0. This is how it cycles back to the start of the range.

Now that our loop is done, we can add one more line outside it to ensure the content is centered. The result should resemble Figure 6-6.

```
slinky.moveTo(500, 500);
```

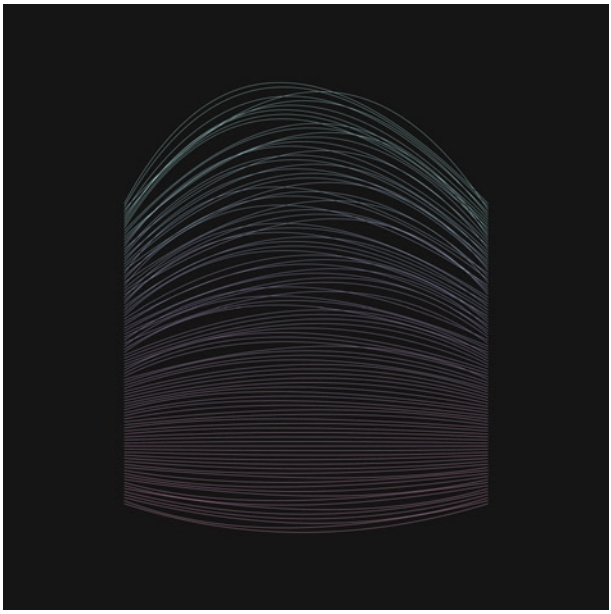

Figure 6-6. *Quadratic curves in a slinky formation*

Elliptical Arcs

Time to embark on the arc! The elliptical arc curve (Figure 6-7) is a complicated beast, and the syntax is quite difficult to comprehend without some visual examples. The `A` (or `a`) command is used to initialize it, and it is followed by no less than seven arguments.

```
// Syntax for the A/a command.
'A [rx ry rotation large-arc-flag sweep-flag x y] ...'
'a [rx ry rotation large-arc-flag sweep-flag dx dy] ...'
```

The first two arguments and the last two arguments aren't anything we haven't encountered before: `rx` and `ry` refer to the radii of the x and y axes, and the final `x` and `y` arguments define the curve's destination point.

The `rotation` defines the angle of rotation along the x axis, and the `large-arc-flag` and `sweep-flag` are both booleans in numeric form, `0` meaning `false` and `1` meaning `true`. What they do though takes a little explaining.

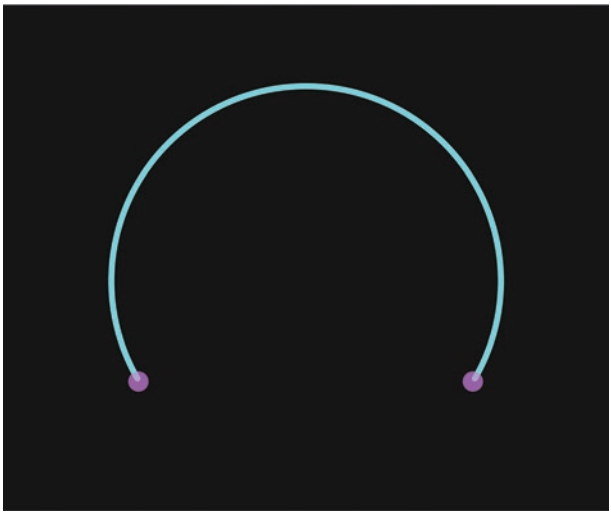

Figure 6-7. *An elliptical arc curve*

Setting the Flags

What's really happening when we draw an elliptical arc curve is that two interlocking ellipses are used as guides behind the scenes to determine which of the many possible paths are actually drawn. With the `large-arc-flag` set to `1`, a large arc will be chosen. And with the `sweep-flag` set to `1`, the arc will be drawn in a clockwise direction. Let's take a look at Figure 6-8, in which four possible permutations of these flags are shown.

- In the uppermost path, the `large-arc-flag` and `sweep-flag` are both set to `1`.
- In the second path down, the `large-arc-flag` is set to `0` and the `sweep-flag` is set to `1`.

- In the third path down, the `large-arc-flag` and the `sweep-flag` are both set to `0`.
- In the lowermost path, the `large-arc-flag` is set to `1` and the `sweep-flag` is set to `0`.

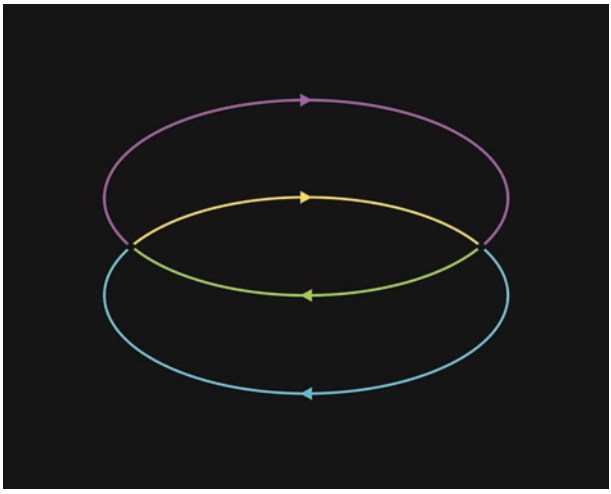

Figure 6-8. *Four possible arcs*

In practice, you'll only ever see one of these arcs rendered on screen. However, it helps us understand the underlying structure of the elliptical arc when we see all the possible curves rendered simultaneously.

Irregular Radii

One of the other potential confusions about the elliptical arc curve is that the two radius values `rx` and `ry` work a little differently to how we might expect. Normally, a radius would directly determine the size of an elliptical shape. In the case of the elliptical arc curve, the size is actually determined by a number of factors that work in conjunction, and the `rx` and `ry` values are only one part of the equation.

You'll sometimes find, for instance, that the interlocking ellipses that form the basis of the possible curves coincide; that is, they are superimposed on one another. You'll know this is the case if setting the `large-arc-flag` to either a `0` or a `1` makes no difference to the final result. This can happen when the two radius values and the distance between the curve start and end points are in a certain proportion to each other.

It might therefore be easier to think of the `rx` and `ry` values as ratios that influence the shape of the arc than as values that directly determine the arc's magnitude (this is especially true when using smaller radii values). Don't worry if this doesn't make much sense – it's quite counterintuitive at first and only becomes clearer with experimentation. And to aid in this experimentation and hopefully make the elliptical arc curve a little more accessible, I've created an interactive pen that shows the effect of altering both the radii and the flag values. You can find the pen at `davidmatthew.ie/generative-art-javascript-svg#arc-curve`.

Generative Arcs

Let's take a break from further wrestling with the theoretical intricacies of elliptical arc curves and put them to some practical, creative use instead. Using our usual copy method, create a new sketch folder and name it something like `15-generative-arcs`. The aim of this sketch will be to create two sets of arc curves: one set running clockwise and the other set mirroring it, running anticlockwise. We'll randomize a number of key variables to keep the output unique and difficult to predict.

When you're ready, set up a group below the background that will contain our arc curves. After that, we'll set up some randomized variables we'll utilize in the loop that follows.

```
// Set up a container group for our arc curves.
let arcs = svg.create('g');
```

```
// Randomise some variables.
let rx = Gen.random(5, 350);
let ry = Gen.random(5, 350);
let hue = Gen.random(0, 360);
```

Next we'll start our loop, which will run 360 times. Inside it, we'll randomize some further variables: the arc's `rotation`, which will have the effect of distending the curve in a particular direction, and the `large-arc-flag`, which, thanks to our `Gen.chance()` function, will have a 50% chance of being either `0` or `1`.

```
for (let i = 0; i < 360; i += 1) {

  // Randomise the rotation and large arc flag.
  let rotation = Gen.random(0, 180);
  let largeArc = Gen.chance() ? 1 : 0;

}
```

Next (and still within our loop) create the first set of clockwise arc curves. Remember, what makes them run either clockwise or anticlockwise is the `sweep-flag`, which is the fifth argument of the `A` command. Here, after our `largeArc` variable, we're setting the `sweep-flag` to `1`, meaning clockwise.

```
// Create a first set of clockwise arc curves (sweep = 1).
arcs.create('path').set({
  fill: 'none',
  stroke: `hsl(${hue} 75% 75% / 0.05)`,
  d: `M 275 500 A ${rx} ${ry} ${rotation} ${largeArc} 1
  725 500`
});
```

The second set of arc curves will be identical, bar two small changes. The `sweep-flag` we'll set to `0`, making this set anticlockwise, and the `hue` we'll offset by `60`, just to add a bit of contrast to the colors. And before we close out the loop, we'll increment the `hue` using the aforementioned modulo operator.

```
// Create a second set of counter-clockwise arc curves (sweep = 0).
arcs.create('path').set({
  fill: 'none',
  stroke: `hsl(${hue + 60} 75% 75% / 0.05)`,
  d: `M 275 500 A ${rx} ${ry} ${rotation} ${largeArc} 0 725 500`
});

// Increment the hue.
hue = (hue % 360) + 0.5;
```

If you run the sketch at this point, you'll see some nicely varied generative arcs (example output in Figure 6-9). If you want to inject a little more dynamism though, you could add the following line after the loop:

```
// Apply a random rotation.
arcs.rotate(Gen.random(0, 360));
```

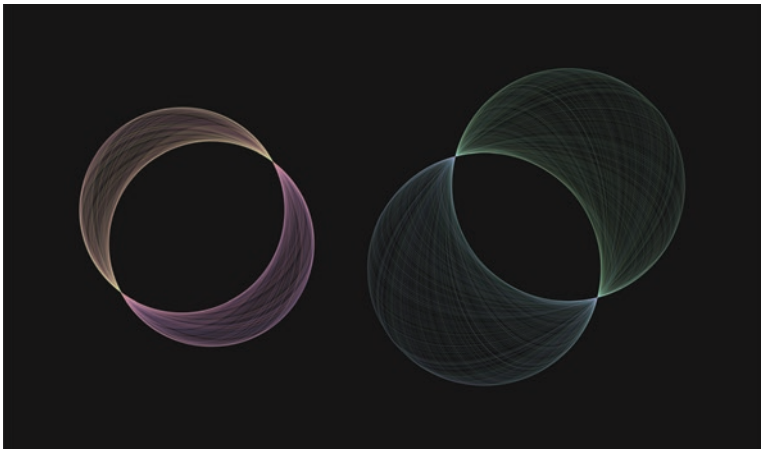

Figure 6-9. *Two renders of our generative arcs*

Cubic Bezier Curves

The final curve we'll cover is the cubic Bezier curve, called by the `C` command. This is the most versatile of all the vector curves available to us and is the main curve utilized in most vector drawing and illustration packages (usually under the alias of the pen tool). It has two sets of control points, so the syntax is a little more complex than its quadratic counterpart, but it's easier to grapple with than the elliptical arc curve.

```
// Syntax of the C/c command.
'C [cpx1 cpy1 cpx2 cpy2 x y] ...'
'c [dcpx1 dcpy1 dcpx2 dcpy2 dx dy] ...'
```

Cubic Control Points

The first set of control point coordinates `cpx1` and `cpy1` shapes the curve from the side of the starting point. This starting point is declared prior to the `C` command – it might be a set of `M` coordinates, for example, or the

end point of a previous curve. The second set of control point coordinates `cpx2` and `cpy2` shapes the curve of the destination point, and this point is defined by the final two coordinates: `x` and `y`. Here's an example:

```
// Creating a single cubic bezier curve.
svg.create('path').set({
  d: 'M 75,75 C 140,330 360,330 425,75'
});
```

The preceding path is illustrated in Figure 6-10, where you can see the initial `M` point in yellow, and in red, the three points that comprise the cubic Bezier curve itself. The colored dots and dashed lines are just for illustrative purposes; these wouldn't be rendered in the final SVG.

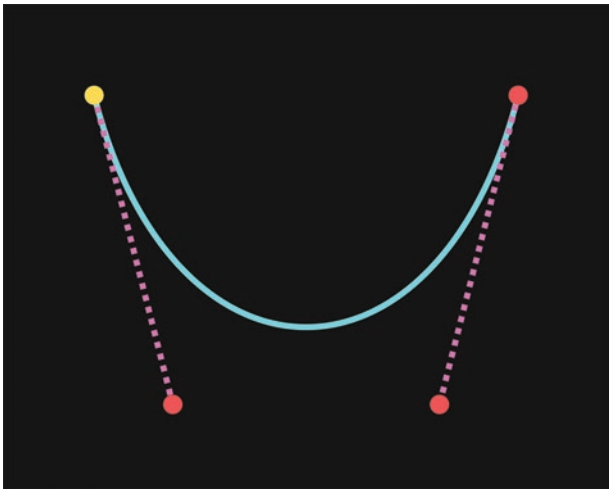

Figure 6-10. *A cubic Bezier curve with the control points visualized*

When linked together, consecutive cubic Bezier curves can combine to create any 2D shape imaginable. They are such a staple of vector drawing programs that wielding them effectively has become a craft in itself. Judicious use of these curves and their "handles" (visualized as the dashed lines in Figure 6-10) can make the difference between smooth, graceful line art and awkward, clunky clip art.

S for Symmetry

Like the quadratic Bezier curve, the cubic Bezier curve has a related command that can be called if you want to extend the curve with another that is smoothly connected to it. The command to use for this is `S` (or `s` for relative values). The `S` command automatically generates a symmetric control point that mirrors the control point of the preceding curve and as a result allows us to omit the first set of control point coordinates.

```
// Extending a cubic bezier curve with the S command.
svg.create('path').set({
  d: 'M 50,150 C 100,250 200,50 250,150 S 400,50 450,150'
});
```

In Figure 6-11, the faintest of the dashed lines indicates where the `S` command has automatically generated a symmetrical control point.

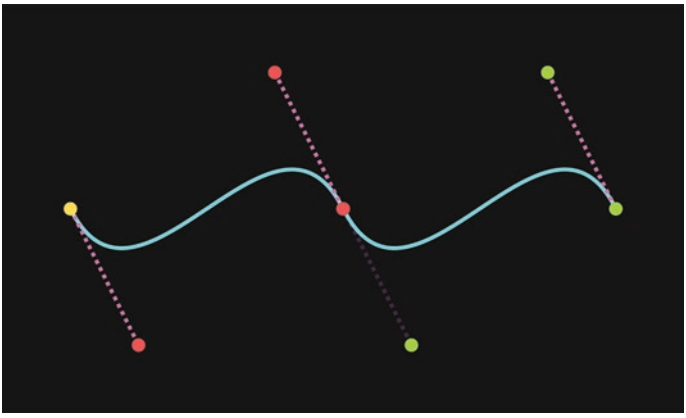

Figure 6-11. *A smoothly extended cubic Bezier curve*

Organic Curves

Let's put the preceding knowledge to use in a composition, the aim of which will be to create a series of smooth, organic-looking curves. To this end, our friend `Noise` will nudge us in the right direction. First, do the usual and copy the template folder, calling it something like `16-organic-curves`. Ensure you have the following imports at the top:

```
import { SvJs, Gen, Noise } from '../../node_modules/svjs/src/
index.js';
```

The parent SVG and background will then follow, at which point we'll create some noise-related variables. I'm including a new one here called `amplifier`; this will be used when we map the noise value to a new range (increasing its volume so to speak).

```
// Noise-related.
let noise = new Noise();
let n = Gen.random(0, 1000);
let speed = 0.05;
let amplifier = Gen.random(200, 500);
```

Next we'll set up variables relating to colors and the curves themselves, which we'll contain within a group.

```
// Curve and colour-related.
let curves = svg.create('g');
let numCurves = Gen.random(75, 125);
let hue = Gen.random(0, 360);
```

It's generally good practice to group your variables meaningfully in this manner, with some comments to add clarity. Otherwise when you later revisit your code, you might have a hard time working out your original intentions. You could go a step further and group them via objects (a technique outlined in Chapter 2), but I won't do that here.

We'll move on to the loop next, where we'll first retrieve and then re-map our noise into a more fit-for-purpose range (using the aforementioned `amplifier` variable). Notice here that the range will span positive and negative values, so we won't know the polarity in advance. In the case of our curves, this will help us mimic a more organic movement, as the values won't always be uniformly offset from the original starting point, but wrap around them more naturally. Outside the loop, include the `moveTo()` method to center-align our curve group.

```
for (let i = 0; i < numCurves; i += 1) {

  // Retrieve and map our noise value.
  let noiseValue = noise.get(n);
  noiseValue = Gen.map(noiseValue, -1, 1, -amplifier,
  amplifier, false);

}

curves.moveTo(500, 500);
```

Next come the curve coordinates. For clarity, we'll set variables for each command we call, to make the construction of the path data (or `d` string) less unwieldy. Continue with the following code, keeping it within the loop. The shape of the curve we're aiming for isn't too dissimilar to the one shown in Figure 6-10. Other than that, the values provided are fairly arbitrary, so feel free to tweak them.

```
// M command co-ordinates.
let mx = 0;
let my = 0 + (i * 5);

// C command co-ordinates.
let cpx1 = 0 + noiseValue;
let cpy1 = -100;
let cpx2 = 250 + noiseValue;
```

```
let cpy2 = -100;
let x2 = 300;
let y2 = 0;

// S command co-ordinates.
let spx = 350 + noiseValue;
let spy = 100;
let x3 = 300;
let y3 = -50;
```

With the exception of the initial `M` command coordinates, we're going to build out our curves using relative coordinates (i.e., we'll feed the preceding curve variables to the `c` and `s` commands rather than to their uppercase brethren). These tend to work better in a loop setting, as they always remain in step with the most recent coordinate values. The only other point to note is that we're being quite selective with which values are being modulated by our `noiseValue` variable, three out of a possible ten in this case. Sometimes subtlety works best, but again, feel free to experiment here.

With our variables in place, it's time to create the curve itself. I'll introduce a new technique here with regard to the use of template literal syntax. In previous sketches, we've been opening and closing our braces `${}` for each variable. When these variables follow one after the other however, we can actually embed them all at once as an array.

```
// Create the organic curve.
curves.create('path').set({
  fill: 'none',
  stroke: `hsl(${hue} 80% 80% / 0.8)`,
  d: `M ${[mx, my]}
      c ${[cpx1, cpy1, cpx2, cpy2, x2, y2]}
      s ${[spx, spy, x3, y3]}`
});
```